

AulaNube: arquitectura de microservicios en la nube para la plataforma de capacitación corporativa de CapacitaPro S.A.S.

[GRUPO N]

Augusto César Cañola Ortiz

[Nombre del integrante]

[Nombre del integrante]

[Nombre del integrante]

Ingeniería de Software, Tecnológica del Oriente

Arquitectura de Software (20262 – N5, Grupo A)

Docente: [Nombre del docente]

Octubre de 2026

Resumen

Este informe presenta la arquitectura propuesta para CapacitaPro S.A.S., empresa ficticia que ofrece capacitación virtual obligatoria a 38 empresas cliente con una instalación de WordPress por cliente, modelo que colapsa en los cierres de campaña, hace esperar al usuario mientras se genera el certificado y multiplica el mantenimiento. A partir del análisis de requisitos se formularon diez escenarios de calidad medibles que orientaron el diseño de AulaNube, una plataforma multiempresa basada en microservicios, con un API Gateway, una base de datos por servicio y eventos asíncronos en Redis Streams. La arquitectura se documentó con diagramas UML de casos de uso, clases, secuencia y despliegue, y se justificaron patrones de diseño para la nube como Transactional Outbox, Queue-Based Load Leveling, Competing Consumers y Circuit Breaker. El prototipo se desplegó en Oracle Cloud Infrastructure, sobre un clúster K3s de dos nodos Ampere A1 de la capa gratuita, y las mediciones confirmaron los escenarios: una campaña masiva responde en menos de 300 ms, el gateway responde en 110 ms con un servicio caído y ningún certificado se pierde cuando falla el trabajador.

Tabla de contenido

Resumen 2
Tabla de contenido 3
AulaNube: arquitectura de microservicios en la nube para la plataforma de capacitación corporativa de CapacitaPro S.A.S. 4
Caso de estudio: CapacitaPro S.A.S. 4
Análisis de requisitos 5
Arquitectura propuesta 8
  Estilo arquitectónico 8
  Relación con los principios de la unidad 9
Diseño de la arquitectura mediante UML 9
  Diagrama de casos de uso 10
  Diagrama de clases 11
  Diagrama de secuencia 12
  Diagrama de despliegue 13
Selección y justificación de patrones de diseño 13
Prototipo funcional en la nube 15
Conclusiones 17
Referencias 18

AulaNube: arquitectura de microservicios en la nube para la plataforma de capacitación corporativa de CapacitaPro S.A.S.

La arquitectura de software es la organización fundamental de un sistema, expresada en sus componentes, sus relaciones y los principios que guían su diseño y evolución (Bass et al., 2012). Como estas decisiones se toman temprano y son costosas de revertir, deben responder a los atributos de calidad que el negocio necesita y no solo a la funcionalidad (Tecnológica del Oriente, 2026).

El informe presenta la solución del equipo para CapacitaPro S.A.S.: el caso y sus requisitos, la arquitectura propuesta, su modelado en UML, los patrones de diseño seleccionados y el prototipo desplegado en la nube con las pruebas que comprueban cada decisión. El código fuente y los diagramas están en https://github.com/augustocco/aulanube, y la aplicación funciona en https://aulanube.149-130-191-139.sslip.io.

Caso de estudio: CapacitaPro S.A.S.

CapacitaPro S.A.S. es una empresa ficticia de Bucaramanga que diseña y administra cursos virtuales que la ley o los contratos hacen obligatorios: inducción al SG-SST, trabajo seguro en alturas, protección de datos personales y atención humanizada en salud. Atiende a 38 empresas cliente y unos 14.000 usuarios; cada cliente tiene su propia instalación de WordPress con un plugin de LMS, repartidas en cinco servidores. La Tabla 1 resume los problemas de ese modelo.

Tabla 1

Problemas de la operación actual de CapacitaPro

Problema	**Evidencia**
Picos en los cierres de campaña	Un cierre de 1.100 trabajadores dejó el servidor 40 minutos sin respuesta y afectó a otros siete clientes.
Certificado dentro de la solicitud	Respuestas de 8 a 25 s al terminar un curso; si el usuario cierra la página, el certificado no se genera.
Correo que bloquea la operación	Cursos terminados que no quedan registrados porque el servidor de correo tardó.
Mantenimiento multiplicado	Cada actualización se aplica 38 veces, cerca del 40 % del tiempo del equipo técnico.
Sin verificación de certificados	Auditores y contratantes piden confirmación por correo, lo que tarda días.

Nota. Elaboración propia a partir del caso de estudio.

La gerencia pide una sola plataforma multiempresa en la que cada cliente vea solo su información, que el cierre de campaña de un cliente no afecte a los demás, certificados verificables por terceros, que la caída de una parte no detenga el resto, crecer por partes, bajo costo durante el piloto y cumplir la Ley 1581 de 2012 (Congreso de la República de Colombia, 2012).

Análisis de requisitos

Se aplicaron cuatro técnicas: análisis documental del caso; una entrevista simulada en la que el equipo representó a la gerencia, a una administradora de empresa cliente y a un colaborador; historias de usuario priorizadas con el método MoSCoW, y escenarios de atributos

de calidad, que expresan cada requisito no funcional con un estímulo y una medida de respuesta verificable (Bass et al., 2012). Las Tablas 2 y 3 presentan el resultado.

Tabla 2

Requisitos funcionales

ID	Historia de usuario	Prioridad
RF01	Como usuario quiero iniciar sesión para acceder a la plataforma de mi empresa.	Debe
RF02	Como estudiante quiero ver los cursos de mi empresa, inscribirme y avanzar por las lecciones.	Debe
RF03	Como estudiante quiero un certificado PDF con código único al completar un curso.	Debe
RF04	Como estudiante quiero un aviso al completar el curso y cuando el certificado esté listo.	Debe
RF05	Como verificador externo quiero consultar un certificado por su código, sin cuenta.	Debe
RF06	Como administrador quiero crear cursos, registrar colaboradores y ver su avance.	Debe
RF07	Como administrador quiero cerrar una campaña masiva sin degradar la plataforma.	Debe
RF08	Como equipo técnico quiero ver la salud de los servicios y la cola de eventos.	Debería

Nota. Cada requisito se traza a un caso de uso de la Figura 2.

Tabla 3

Requisitos no funcionales expresados como escenarios de calidad

ID	Atributo	Escenario y medida de aceptación
RNF01	Escalabilidad	150 cursos terminados en un minuto: respuesta < 1 s y todos los certificados emitidos.
RNF02	Rendimiento	Respuesta < 500 ms al completar el curso; certificado disponible en < 10 s.
RNF03	Disponibilidad	Con certificados caído, los demás servicios responden y el gateway contesta en < 300 ms.
RNF04	Confiabilidad	Si Redis o un trabajador fallan, no se pierden certificados ni se duplican.
RNF05	Seguridad	Una empresa no lee datos de otra; los servicios internos no son accesibles desde fuera.
RNF06	Equidad	Una empresa que supera 20 solicitudes por segundo recibe 429 sin afectar a las demás.
RNF07	Modificabilidad	Cambiar el envío de avisos solo modifica el servicio de notificaciones.
RNF08	Costo	La infraestructura del piloto se mantiene en la capa gratuita del proveedor.

Nota. Elaboración propia con el formato de escenarios de calidad de Bass et al. (2012).

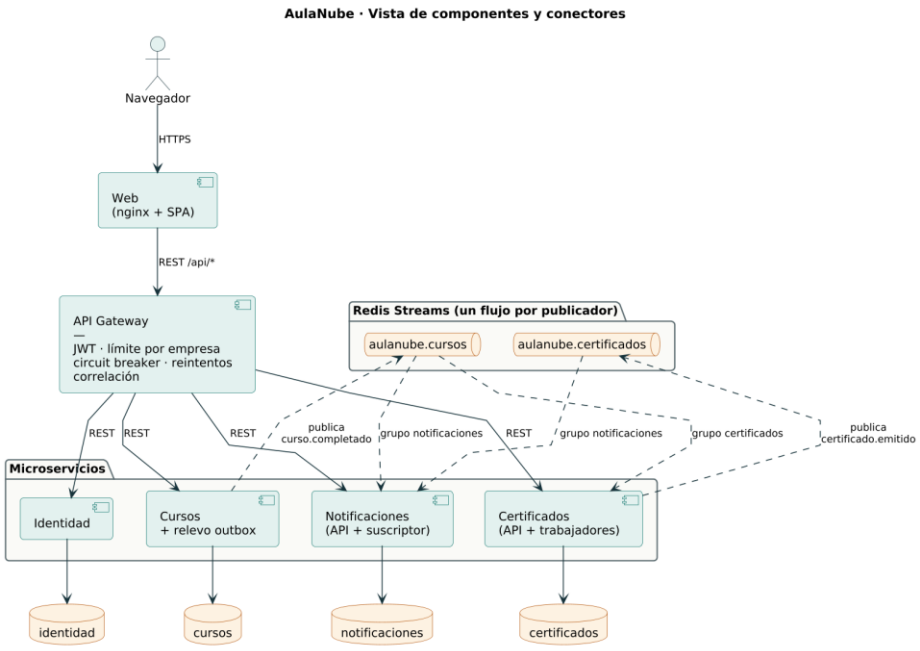
Arquitectura propuesta

Estilo arquitectónico

Se eligió una arquitectura de microservicios con comunicación híbrida: REST síncrono para lo que el usuario consulta de inmediato y eventos asíncronos para lo que puede ocurrir después, como emitir el certificado y avisar. Los escenarios RNF01 y RNF03 exigen escalar y aislar fallos por partes; un monolito modular, aunque más sencillo, escala y falla como una sola unidad, que es precisamente el problema de CapacitaPro (Red Hat, 2023; Redwerk, 2025). La Figura 1 muestra la web, el API Gateway como única entrada y cuatro servicios con base de datos propia: identidad, cursos, certificados y notificaciones.

Figura 1

Vista de componentes y conectores de AulaNube



Nota. Elaboración propia. Las flechas continuas son llamadas REST; las punteadas, eventos en Redis Streams.

Relación con los principios de la unidad

Hay **alta cohesión** porque cada servicio agrupa un solo concepto del dominio. Hay **bajo acoplamiento** porque los servicios no se llaman entre sí: cursos publica `curso.completado` sin saber quién lo consume, de modo que agregar facturación no exige modificarlo. Y se respeta el **ocultamiento de información** de Parnas (1972): la base de cada servicio es un detalle interno, por lo que trasladar los PDF a un almacenamiento de objetos no afectaría a otro módulo. Como compromisos se aceptaron la consistencia eventual del certificado, una sola instancia de PostgreSQL con cuatro bases separadas y Redis Streams en lugar de un bróker dedicado.

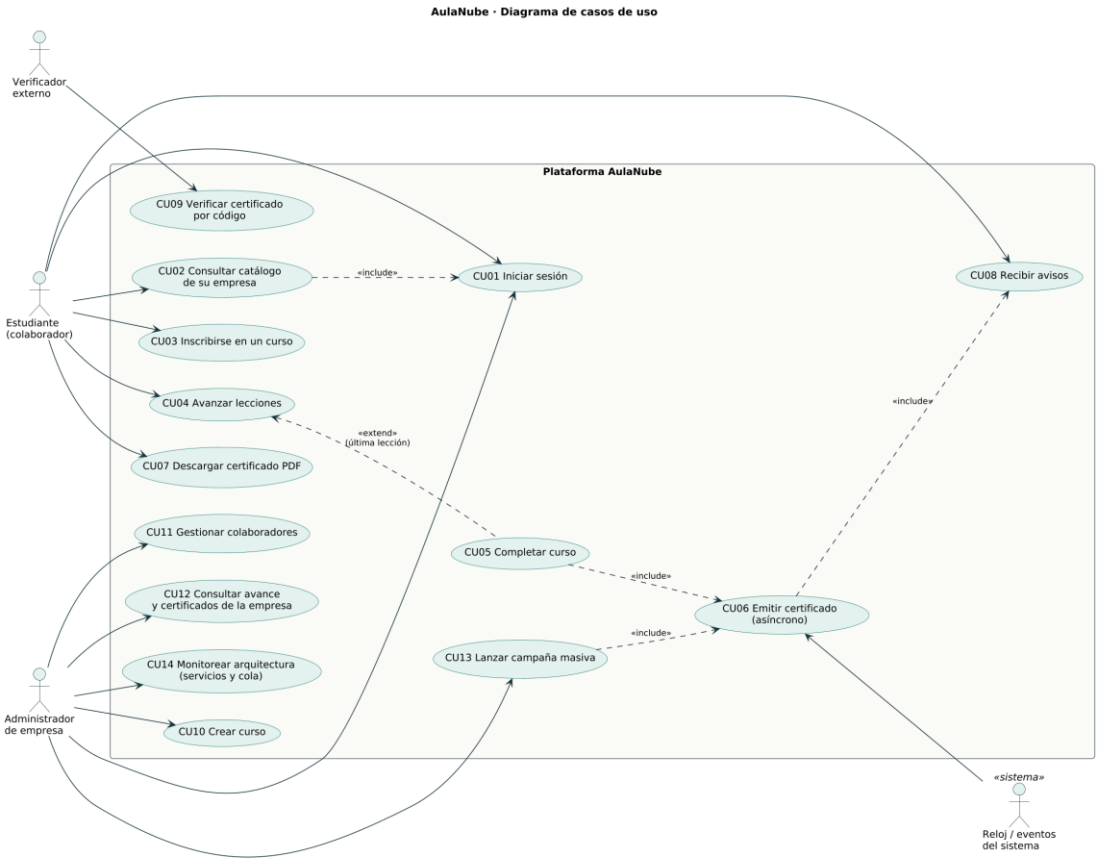
Diseño de la arquitectura mediante UML

Los diagramas se elaboraron en PlantUML con la especificación UML 2.5.1 (Object Management Group, 2017); su código fuente está en la carpeta `docs/uml` del repositorio.

Diagrama de casos de uso

Figura 2

Diagrama de casos de uso de AulaNube



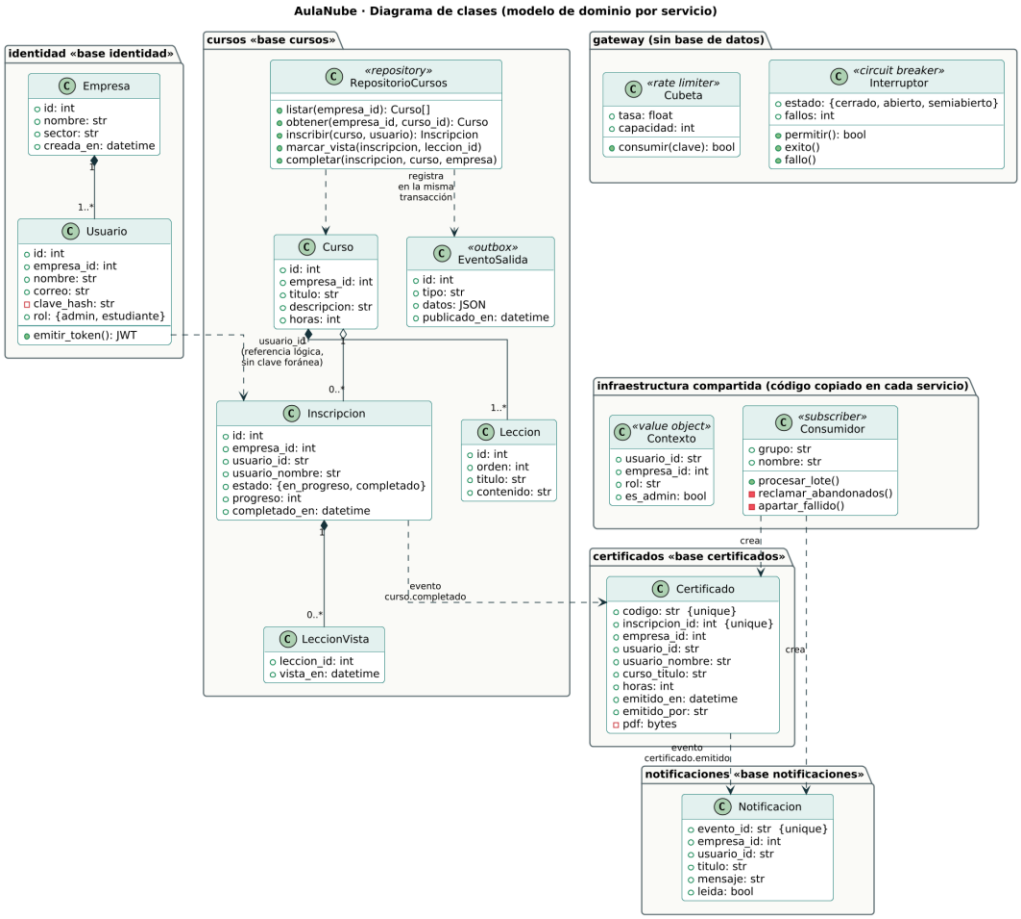
Nota. Elaboración propia.

Participan el estudiante, el administrador de la empresa, un verificador externo sin cuenta y el sistema. «Completar curso» incluye «Emitir certificado», que a su vez incluye «Recibir avisos». Es relevante porque la emisión la dispara un evento y no una persona, lo que la señala como asíncrona, y porque el verificador obliga a una ruta pública controlada en el gateway.

Diagrama de clases

Figura 3

Modelo de dominio por servicio



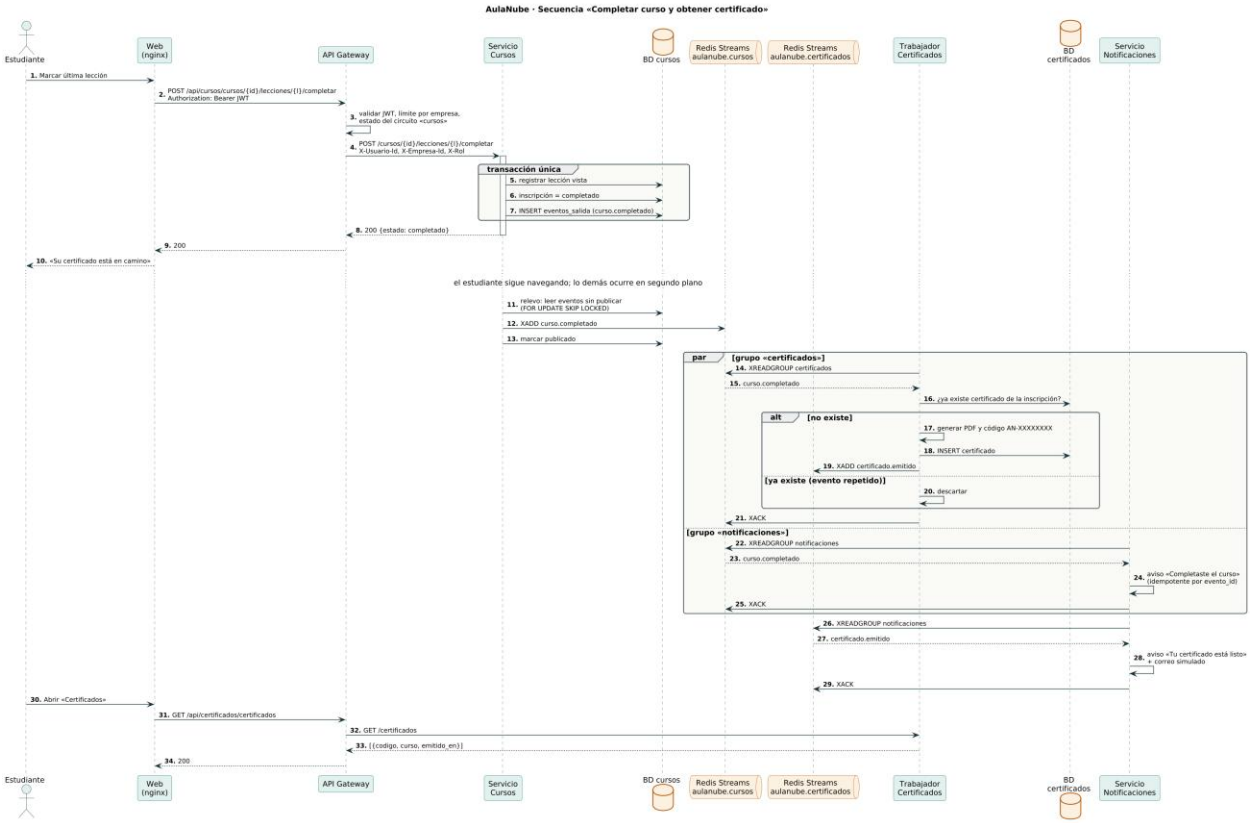
Nota. Elaboración propia. Cada paquete corresponde a un servicio y a su base de datos.

Ninguna asociación cruza un paquete: entre servicios solo hay referencias lógicas o eventos, lo que comprueba la decisión de una base por servicio. `RepositorioCursos` impone el filtro por empresa, y las restricciones únicas de certificado y notificación evitan duplicados cuando un evento llega dos veces.

Diagrama de secuencia

Figura 4

Secuencia para completar un curso y obtener el certificado



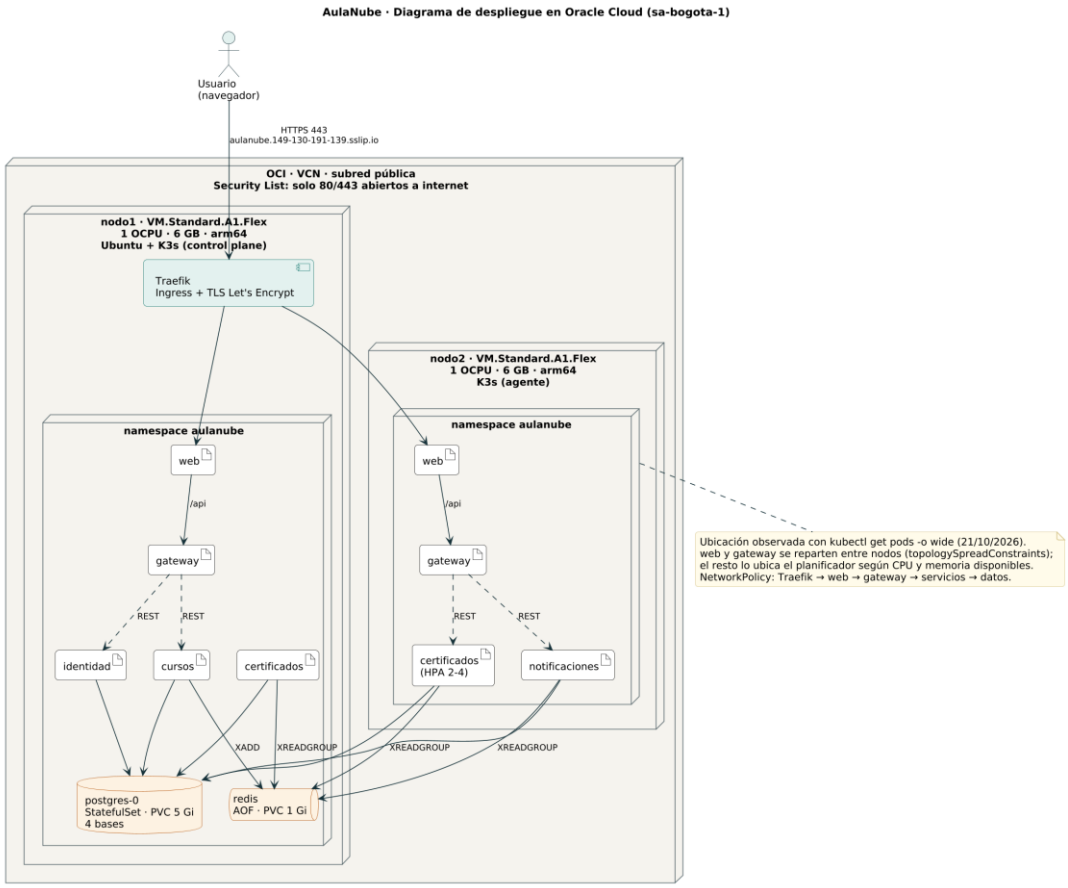
Nota. Elaboración propia.

Cursos guarda el avance y el evento en la misma transacción y responde de inmediato; luego un hilo publica el evento en `aulanube.cursos`. Certificados y notificaciones lo reciben en paralelo; el trabajador genera el PDF y publica `certificado.emitido`. Así el usuario no espera (RNF02), el pico queda en la cola (RNF01) y ningún evento se pierde si Redis falla (RNF04).

Diagrama de despliegue

Figura 5

Despliegue en Oracle Cloud Infrastructure, región sa-bogota-1



Nota. Elaboración propia a partir de los manifiestos de Kubernetes y de la ubicación de los pods observada el 21 de octubre de 2026.

Dos instancias Ampere A1 de 1 OCPU y 6 GB forman un clúster K3s en la capa gratuita de Oracle (s. f.). Traefik termina TLS; la web y el gateway tienen una réplica por nodo; certificados escala de dos a cuatro réplicas, y las políticas de red solo permiten el camino web, gateway, servicios y datos.

Selección y justificación de patrones de diseño

Los patrones se seleccionaron a partir de los escenarios de calidad y del catálogo de patrones para la nube de Microsoft (s. f.) y Richardson (2018). Blas et al. (2019) señalan que

deben modelarse y verificarse como parte de la arquitectura; por eso cada uno aparece en los diagramas y se comprueba en el prototipo (Tabla 4).

Tabla 4

Patrones de diseño seleccionados

Patrón	**Aplicación en AulaNube**	**RNF**
API Gateway	Única entrada; valida el token y reenvía la identidad del usuario.	05
Database per Service	Una base y un usuario por servicio.	07
Publisher/Subscriber	Un flujo de Redis por publicador y un grupo por suscriptor.	07
Transactional Outbox	El evento se guarda con el cambio y se publica después.	04
Queue-Based Load Leveling	La campaña responde 202; la cola absorbe el pico.	01
Competing Consumers	Varias réplicas comparten la cola de certificados.	01
Circuit Breaker	Tras tres fallos el circuito se abre 15 s y responde de inmediato.	03
Rate Limiting	20 solicitudes por segundo por empresa.	06
Repository	Filtro por empresa en un solo lugar (Fowler, 2002).	05

Nota. Elaboración propia. La columna RNF indica el escenario de la Tabla 3 que atiende cada patrón.

Circuit Breaker responde al correo que bloqueaba la finalización del curso: convierte un fallo lento en uno rápido antes de que agote las conexiones del llamador (Nygard, 2018). La bandeja de salida resuelve que guardar y publicar no pueden ser atómicos, y la idempotencia de los consumidores cubre la entrega «al menos una vez» (Richardson, 2018). Se descartaron Saga,

porque ningún flujo exige deshacer pasos en varios servicios, y un service mesh, que excede los nodos de 1 OCPU.

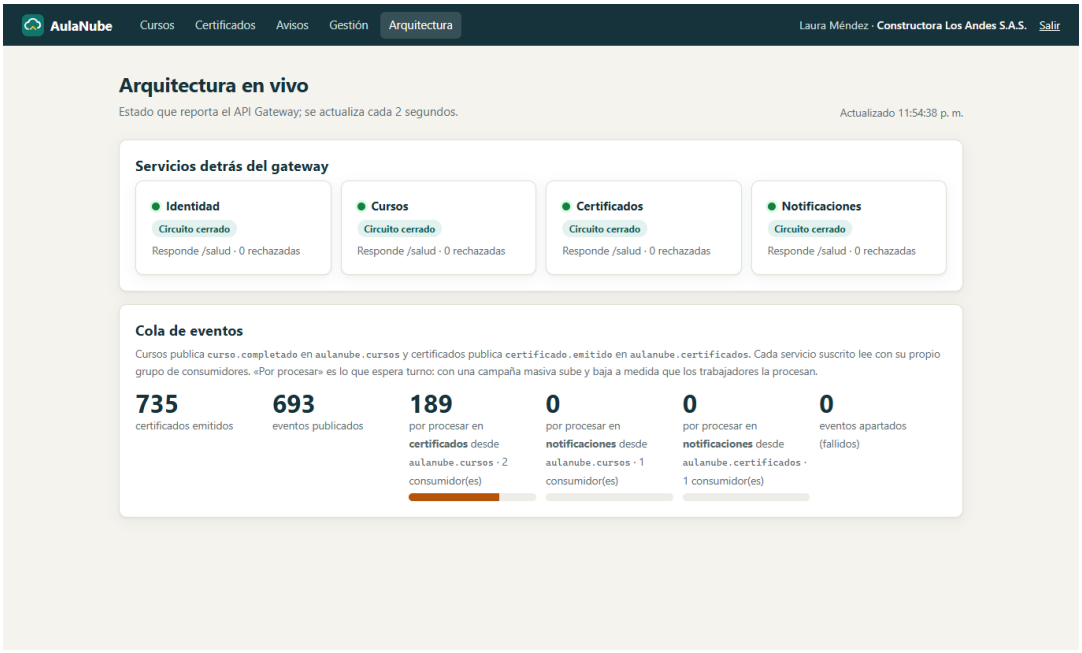
Prototipo funcional en la nube

Los servicios están escritos en Python con FastAPI; la interfaz se sirve con nginx; los datos residen en PostgreSQL 16 y los eventos en Redis 7.4. El sistema corre en local con Docker Compose y en la nube con cinco manifiestos de Kubernetes. Las 28 pruebas automatizadas pasan en x86 y en arm64, y una prueba de humo recorre el flujo completo contra el despliegue.

En una campaña de 200 participantes el servicio respondió en menos de 300 ms y dos réplicas vaciaron la cola a unos 6,5 certificados por segundo (Figura 6). Con certificados apagado, el gateway respondió en unos 110 ms con el circuito abierto mientras los cursos seguían funcionando (Figura 7), y un curso completado durante la caída recibió su certificado nueve segundos después de recuperar el servicio.

Figura 6

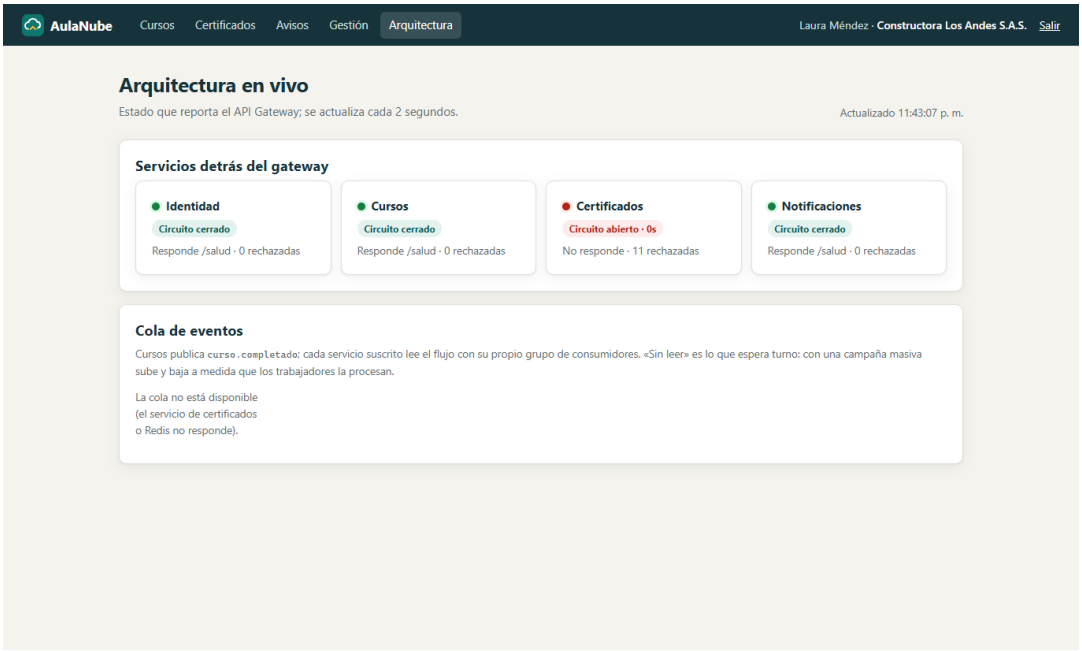
Panel de arquitectura durante una campaña de 200 participantes



Nota. Captura del 21 de octubre de 2026 en el despliegue de Oracle Cloud Infrastructure.

Figura 7

Circuito del servicio de certificados abierto



Nota. Captura del 21 de octubre de 2026. Los demás servicios permanecen disponibles.

Tabla 5

Resultados medidos frente a los escenarios de calidad

RNF	Medida esperada	Resultado medido
RNF01	Campaña masiva < 1 s	195–300 ms; 200 certificados sin fallos
RNF02	Respuesta < 500 ms	Unos 150 ms; certificado en 1–2 s
RNF03	< 300 ms con un servicio caído	110–150 ms con el circuito abierto
RNF04	Ningún certificado perdido	Evento procesado al recuperar el servicio
RNF08	Capa gratuita	Dos nodos de 1 OCPU dentro de Always Free

Nota. Mediciones del 21 de octubre de 2026 sobre el despliegue en la nube.

Conclusiones

Los problemas de CapacitaPro eran de arquitectura y no de código: al cambiar la estructura, los síntomas desaparecieron sin optimizar la lógica de negocio, lo que confirma que las decisiones estructurales tempranas determinan la calidad del sistema (Bass et al., 2012).

Expresar los requisitos no funcionales como escenarios medibles permitió elegir los patrones con criterio y comprobarlos, y la cohesión, el acoplamiento y el ocultamiento de información se volvieron decisiones verificables. La asincronía tiene costos que deben hacerse explícitos, como la consistencia eventual y los consumidores idempotentes. Por último, el prototipo completo opera en la capa gratuita de Oracle con TLS, escalado automático y segmentación de red.

Referencias

Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3.ª ed.). Addison-Wesley.

Blas, M. J., Leone, H. P., & Gonnet, S. M. (2019). Modelado y verificación de patrones de diseño de arquitectura de software para entornos de computación en la nube. En *Proceedings del Congreso Nacional de Ingeniería Informática/Sistemas de Información*.

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial n.º 48.587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Microsoft. (s. f.). *Cloud design patterns*. Azure Architecture Center. Recuperado el 20 de octubre de 2026, de https://learn.microsoft.com/en-us/azure/architecture/patterns/

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2.ª ed.). Pragmatic Bookshelf.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Oracle. (s. f.). *Always Free resources*. Oracle Cloud Infrastructure Documentation. Recuperado el 20 de octubre de 2026, de https://docs.oracle.com/en-us/iaas/Content/FreeTier/freetier*topic-Always*Free_Resources.htm

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM, 15*(12), 1053–1058. https://doi.org/10.1145/361598.361623

Red Hat. (2023). *¿Qué son los microservicios? Ventajas de la arquitectura de microservicios.* https://www.redhat.com/es/topics/microservices/what-are-microservices

Redwerk. (2025). *Arquitectura monolítica frente a la de microservicios para .NET*. https://redwerk.es/blog/arquitectura-monolitica-vs-microservicios/

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.

Tecnológica del Oriente. (2026). *Arquitectura de software: Principios, patrones y evaluación. Unidad 1: Principios de arquitectura de software* [Material del curso Arquitectura de Software]. Institución de Educación Superior Tecnológica del Oriente.